BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN




**BÁO CÁO ĐỒ ÁN CUỐI KỲ**
**MÔN ĐIỆN TOÁN ĐÁM MÂY**

# XỬ LÝ ẢNH TỰ ĐỘNG:
# TẠO THUMBNAIL KHI UPLOAD LÊN S3

**HỌC KỲ 1 – NĂM HỌC 2026 – 2027**

**GVHD: TS. Huỳnh Xuân Phụng**

Sinh viên thực hiện:



| HỌ VÀ TÊN | MSSV |
|---|---|
| Bùi Nguyễn Duy Trung | 24110363 |
| Đỗ Anh Tuấn | 24110369 |



*Thành phố Hồ Chí Minh, tháng 10 năm 2026*

# MỤC LỤC

**CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI ..... 1**

1.1. Đặt vấn đề ..... 1

1.2. Mục tiêu và phạm vi ..... 1

**CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG ..... 2**

2.1. Kiến trúc tổng thể ..... 2

2.2. Tổ chức dữ liệu và quyền truy cập ..... 2

**CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ TÀI NGUYÊN ..... 3**

3.1. Chuẩn bị công cụ và kiểm thử cục bộ ..... 3

3.2. Khởi tạo môi trường AWS Academy ..... 3

3.3. Kiểm tra role dùng cho dịch vụ xử lý ảnh ..... 4

3.4. Tạo vùng lưu ảnh gốc và ảnh thu nhỏ ..... 5

3.5. Cấu hình bảo vệ dữ liệu ảnh ..... 6

3.5.1. Chặn truy cập công khai ..... 6

3.5.2. Quản lý quyền sở hữu đối tượng ..... 7

3.5.3. Mã hóa và gắn nhãn tài nguyên ..... 8

3.6. Tạo bảng lưu thông tin xử lý ảnh ..... 9

3.7. Chuẩn bị thư viện tạo thumbnail ..... 10

3.8. Theo dõi ngân sách thực hành ..... 10

**CHƯƠNG 4. TỔ CHỨC THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ ..... 12**

4.1. Phân công công việc ..... 12

4.2. Kết quả đạt được ..... 12

4.3. Đánh giá và kết luận ..... 13

4.4. Khai báo sử dụng AI và minh chứng hoàn thiện ..... 14

4.4.1. ChatGPT — Hỗ trợ mã nguồn, hướng dẫn cấu hình và rà soát báo cáo 14

4.4.2. Google Gemini — Tóm lược, rà soát commit và hướng dẫn trình bày Markdown ..... 15

4.4.3. Danh mục ảnh cấu hình và phạm vi minh chứng ..... 16

**TÀI LIỆU THAM KHẢO ..... 17**

# CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

## 1.1. Đặt vấn đề

Trong website bán hàng hoặc thư viện ảnh, một ảnh gốc thường cần nhiều phiên bản kích thước nhỏ để hiển thị ở danh sách, trang chi tiết và phần xem trước. Việc tạo các phiên bản này bằng tay mất thời gian và khó bảo đảm tính thống nhất khi số lượng ảnh tăng.

Đề tài "Xử lý ảnh tự động: tạo thumbnail khi upload lên S3" xây dựng quy trình tiếp nhận ảnh gốc trên Amazon S3 và tự động tạo ảnh thu nhỏ (thumbnail). Quy trình dự kiến sử dụng AWS Lambda kết hợp thư viện Pillow để xử lý ảnh, lưu kết quả vào S3 và quản lý thông tin xử lý bằng DynamoDB.

## 1.2. Mục tiêu và phạm vi

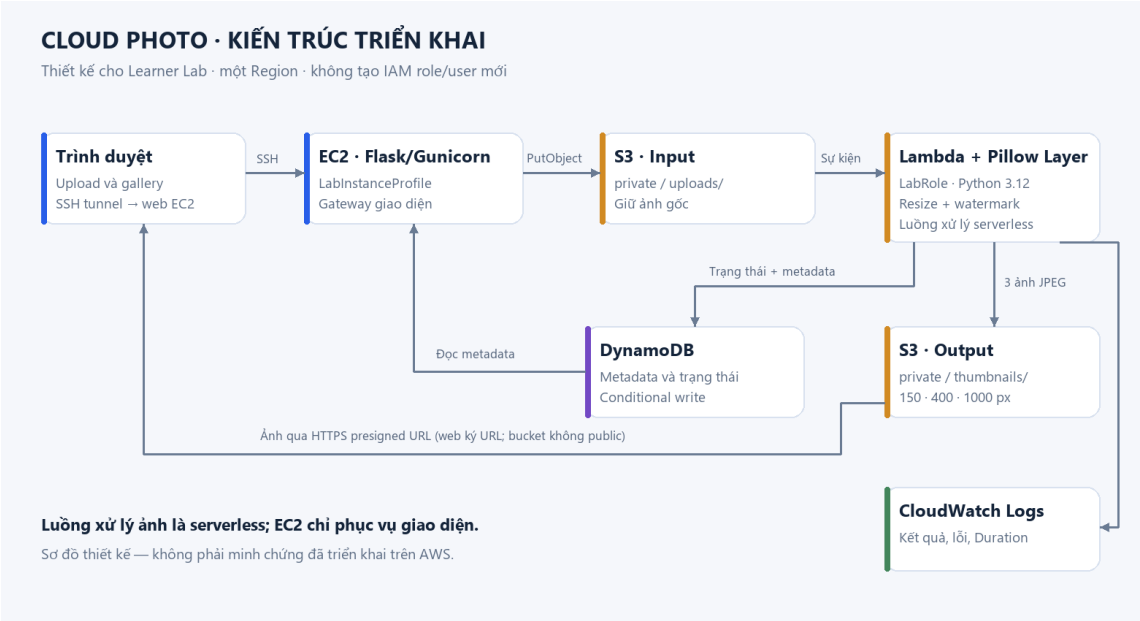
Hệ thống hướng đến tiếp nhận ảnh JPG/PNG, tạo ba phiên bản JPEG có cạnh dài tối đa 150, 400 và 1000 pixel, giữ nguyên tỷ lệ và bổ sung watermark. Ảnh gốc và ảnh kết quả được lưu riêng để thuận tiện quản lý và tránh kích hoạt xử lý lặp.

Phạm vi đầu vào là ảnh tĩnh, dung lượng tối đa 10 MiB và không quá 24 triệu pixel. Ảnh nhỏ không bị phóng to; ảnh PNG có nền trong suốt được ghép nền trắng khi chuyển sang JPEG. Giao diện Flask được thiết kế để hỗ trợ tải ảnh và xem kết quả.

Trong tuần 1, mục tiêu thực hiện là xác định kiến trúc, chuẩn bị môi trường phát triển, thiết lập hai bucket S3, bảng DynamoDB và Pillow Layer, đồng thời kiểm tra mã nguồn bằng bộ kiểm thử cục bộ.

# CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

## 2.1. Kiến trúc tổng thể




*Hình 2.1. Kiến trúc hệ thống xử lý ảnh tự động trên AWS*

Theo thiết kế, người dùng tải ảnh vào bucket đầu vào. Sự kiện tạo đối tượng trên S3 sẽ kích hoạt Lambda; hàm xử lý sử dụng Pillow để tạo thumbnail và watermark, sau đó ghi ảnh vào bucket đầu ra. DynamoDB lưu thông tin ảnh và trạng thái xử lý, còn CloudWatch lưu nhật ký để theo dõi lỗi. EC2 và Flask phục vụ giao diện tải ảnh, xem kết quả.

## 2.2. Tổ chức dữ liệu và quyền truy cập

Bucket đầu vào chứa ảnh gốc trong thư mục uploads/. Bucket đầu ra dành cho ba kích thước thumbnail của mỗi ảnh. Việc tách hai bucket giúp ảnh kết quả không đi vào nguồn kích hoạt của quy trình xử lý.

Bảng cloud-photo-metadata sử dụng khóa chính image_id kiểu chuỗi để nhận diện từng ảnh. Các thông tin dự kiến lưu gồm vị trí ảnh gốc, tên tệp, trạng thái, kích thước ảnh, danh sách thumbnail và thời gian xử lý. Trạng thái được thiết kế gồm đang xử lý, hoàn thành, bị từ chối và thất bại.

Các bucket sử dụng chế độ truy cập riêng tư và mã hóa phía máy chủ. Ứng dụng sẽ truy cập tài nguyên theo quyền được cấp và cung cấp URL ký sẵn khi cần hiển thị ảnh. Pillow Layer được tách riêng để cung cấp thư viện xử lý ảnh cho Lambda.

# CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ TÀI NGUYÊN

## 3.1. Chuẩn bị công cụ và kiểm thử cục bộ

Nhóm cài đặt Python để chạy mã xử lý ảnh và Git để quản lý mã nguồn. Sau khi cài đặt, nhóm kiểm tra phiên bản bằng lệnh trong Terminal. Kết quả xác nhận Python 3.12.10 và Git 2.54.0.windows.1 đã sẵn sàng trên máy phát triển.

```
PS D:\Subject_K5\CloudComputing\CloudComputing_Final> py -3.12 --version
Python 3.12.10
PS D:\Subject_K5\CloudComputing\CloudComputing_Final> git --version
git version 2.54.0.windows.1
```

*Hình 3.1. Kết quả kiểm tra phiên bản Python và Git*

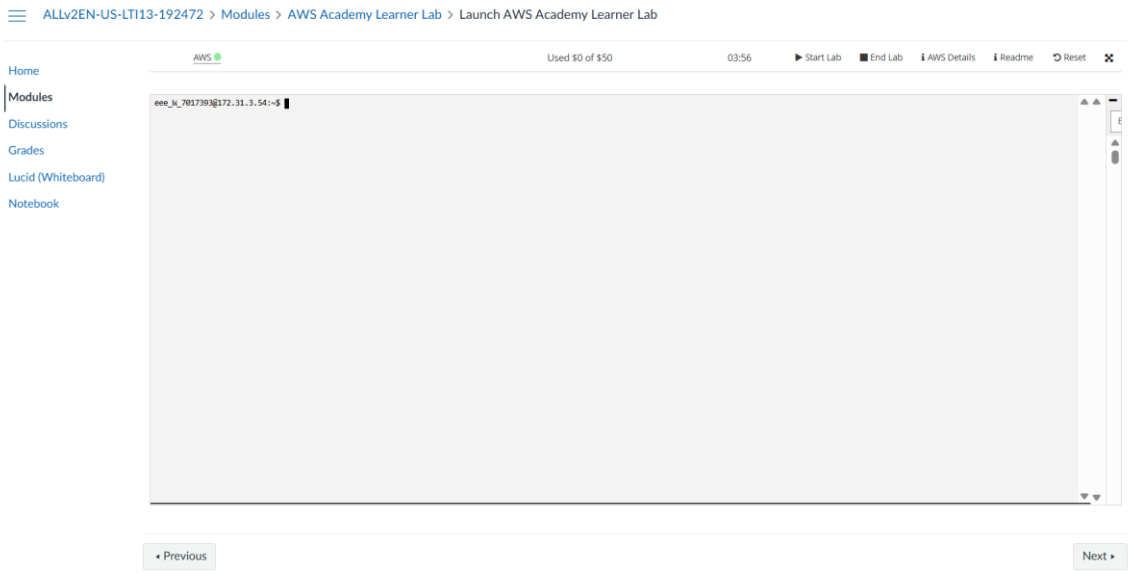
Nhóm tạo môi trường ảo, cài các thư viện của dự án và chạy bộ kiểm thử bằng pytest. Kết quả có 19/19 kiểm thử đạt trong 3,00 giây. Bộ kiểm thử kiểm tra mã xử lý ảnh, worker và giao diện với các thành phần AWS được giả lập; kết quả này là cơ sở chuẩn bị cho bước tích hợp trên AWS.

```
PS D:\Subject_K5\CloudComputing\CloudComputing_Final> .\.venv\Scripts\python.exe -m pytest tests -q
...................                                                      [100%]
19 passed in 3.00s
```

*Hình 3.2. Kết quả chạy bộ kiểm thử cục bộ*

## 3.2. Khởi tạo môi trường AWS Academy

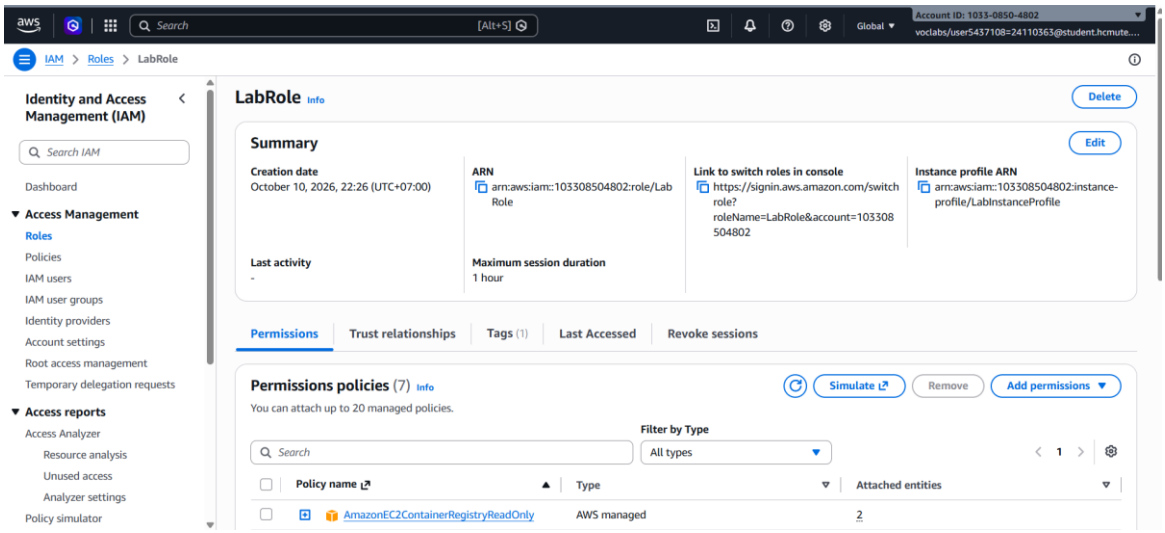
Nhóm khởi động AWS Academy Learner Lab bằng Start Lab và chờ trạng thái AWS chuyển sang màu xanh. Sau đó, nhóm truy cập AWS Console và sử dụng tài khoản 103308504802, Region us-east-1 (N. Virginia) để tạo các tài nguyên của đề tài. Việc thống nhất Region giúp các thành phần lưu trữ và xử lý ảnh được cấu hình nhất quán.




*Hình 3.3. Môi trường AWS Academy Learner Lab đã sẵn sàng*

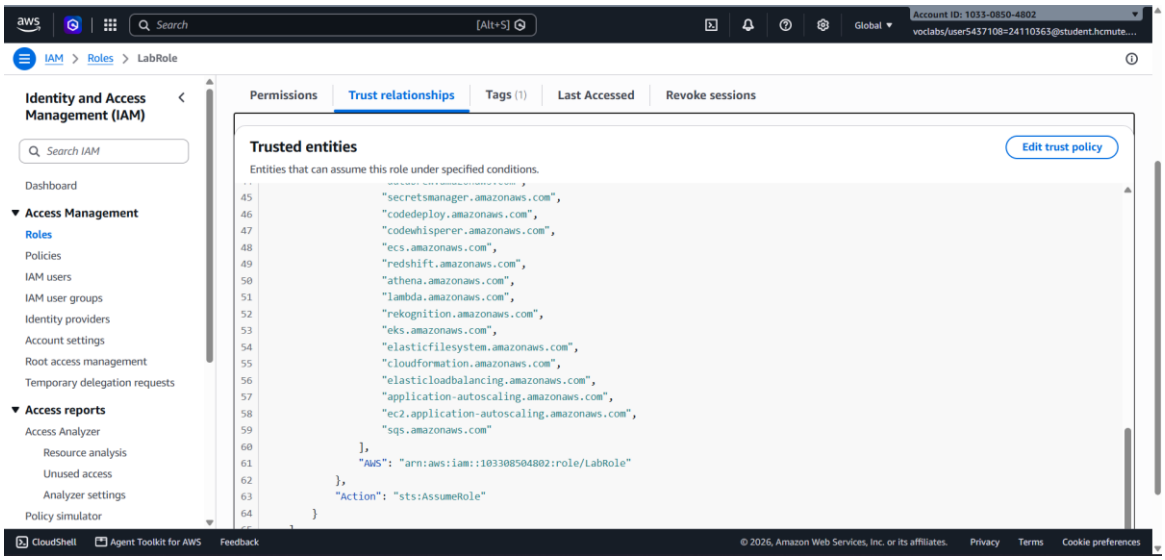
### 3.3. Kiểm tra role dùng cho dịch vụ xử lý ảnh

Trong IAM, nhóm mở mục Roles và kiểm tra LabRole có sẵn của môi trường thực hành. Màn hình tổng quan xác nhận role thuộc tài khoản đang sử dụng và có cấu hình LabInstanceProfile. LabRole được lựa chọn để cấp quyền thực thi cho Lambda khi triển khai hàm xử lý ảnh.




*Hình 3.4. Thông tin tổng quan của LabRole*

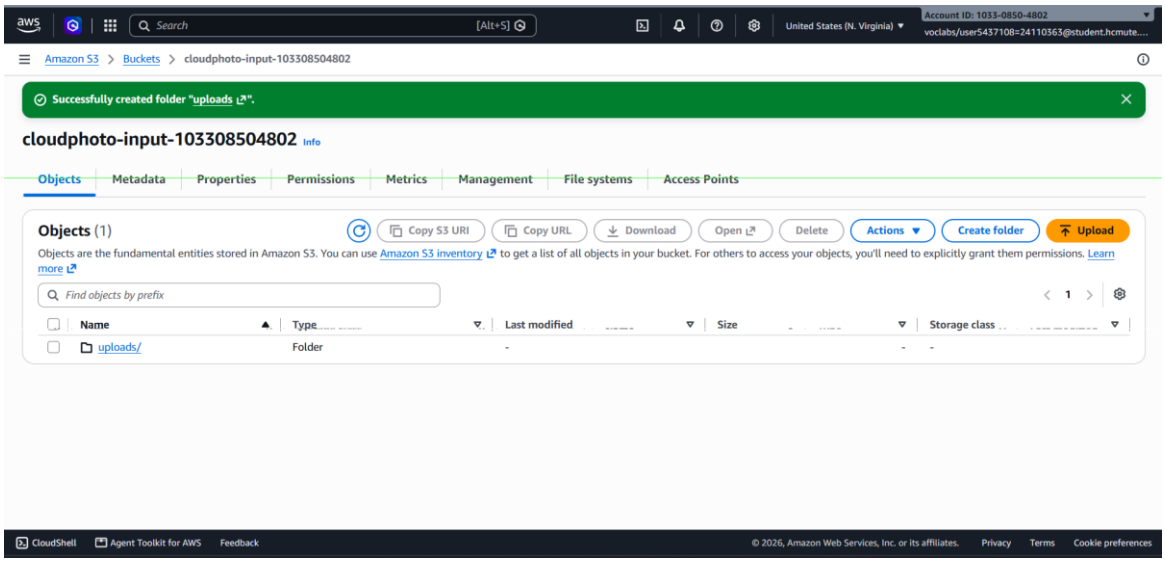
Tiếp theo, nhóm kiểm tra mục Trust relationships. Chính sách có dịch vụ lambda.amazonaws.com và hành động sts:AssumeRole, cho phép Lambda đảm nhận role theo chính sách tin cậy. Đây là bước chuẩn bị về danh tính thực thi; quyền đọc, ghi tài nguyên sẽ được kiểm tra khi tích hợp hàm.




*Hình 3.5. Chính sách tin cậy của LabRole có dịch vụ Lambda*

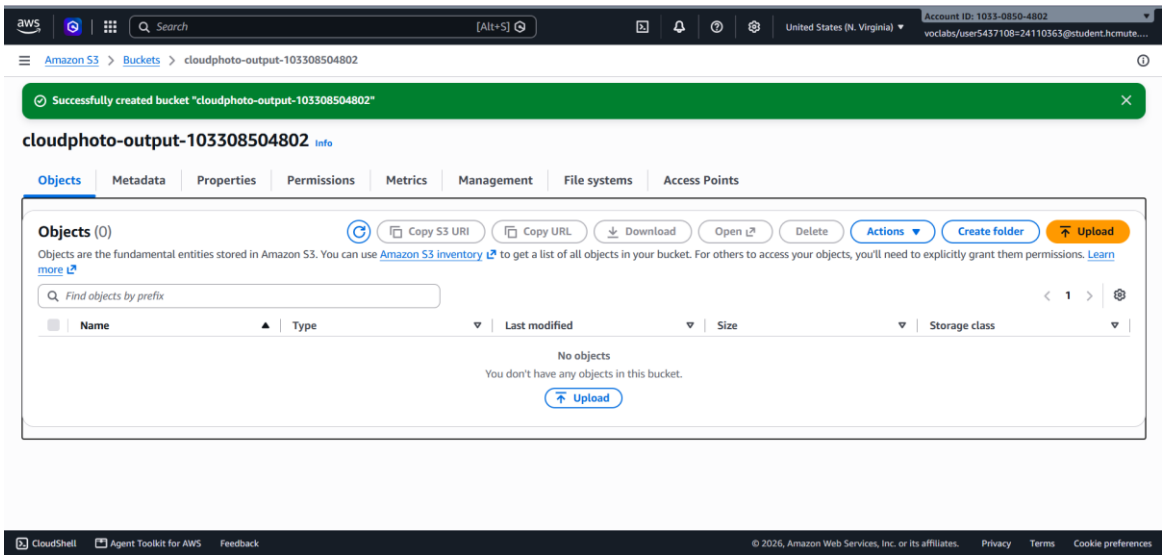
## 3.4. Tạo vùng lưu ảnh gốc và ảnh thu nhỏ

Nhóm tạo bucket cloudphoto-input-103308504802 trong Amazon S3, sau đó dùng Create folder để tạo thư mục uploads/. Đây là vị trí tiếp nhận ảnh gốc và cũng là tiền tố dự kiến được cấu hình cho sự kiện kích hoạt xử lý ảnh.




*Hình 3.6. Bucket đầu vào đã có thư mục uploads/*

Bucket cloudphoto-output-103308504802 được tạo cùng Region để lưu các ảnh thu nhỏ do Lambda sinh ra. Tại thời điểm hoàn thành bước chuẩn bị, bucket chưa có đối tượng vì quy trình tạo thumbnail chưa được tích hợp.

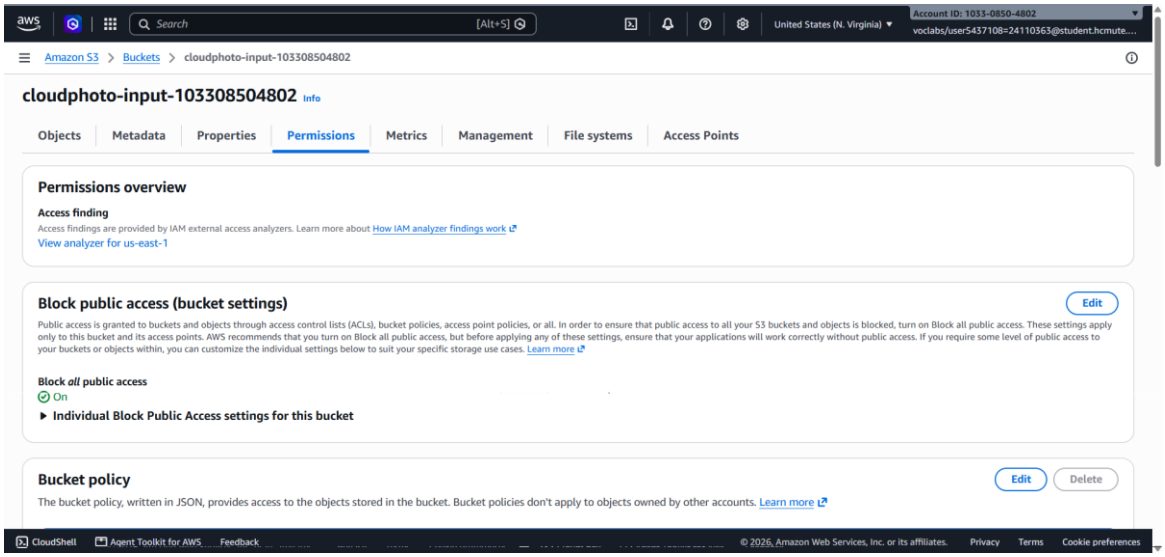



*Hình 3.7. Bucket đầu ra đã được tạo để lưu thumbnail*

## 3.5. Cấu hình bảo vệ dữ liệu ảnh

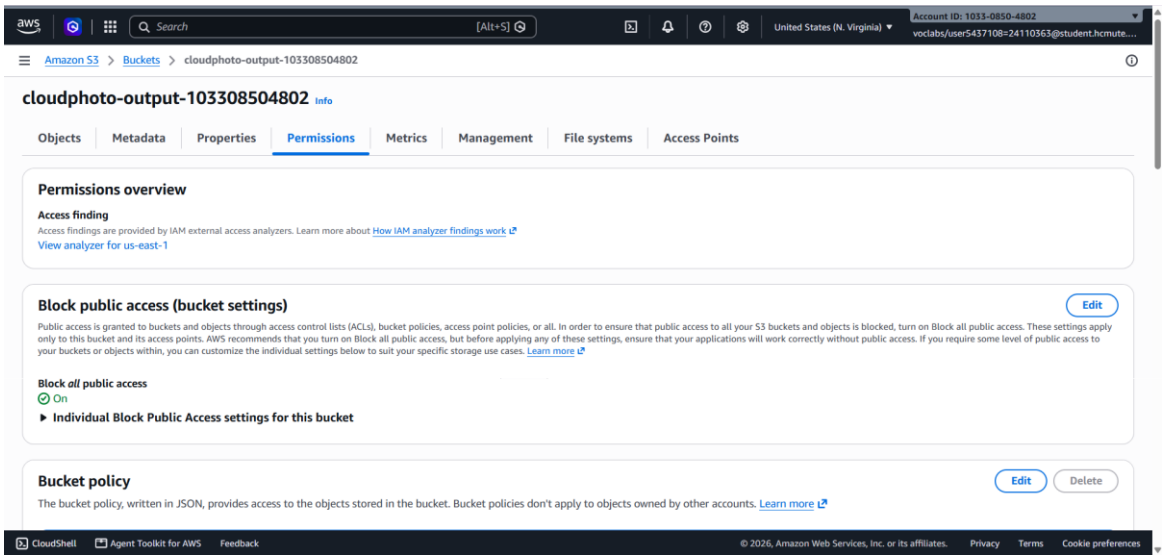
### 3.5.1. Chặn truy cập công khai

Nhóm giữ Block all public access ở trạng thái On trong mục Permissions của cả hai bucket. Với bucket đầu vào, cấu hình này giúp ngăn việc công khai ảnh gốc thông qua các thiết lập truy cập công khai trên S3.




*Hình 3.8. Bucket đầu vào đã bật chặn truy cập công khai*

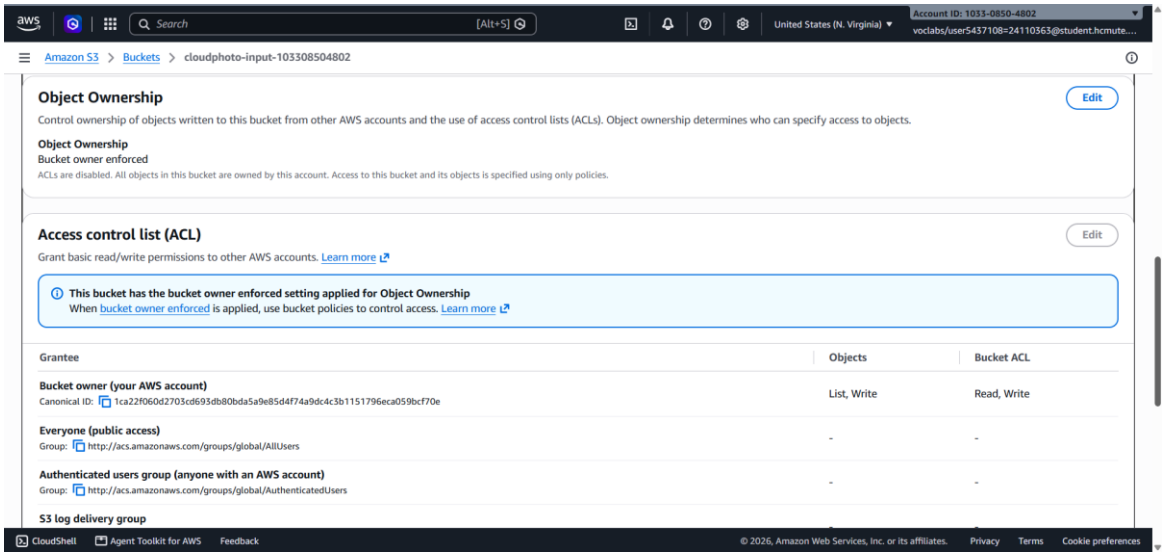
Bucket đầu ra áp dụng cùng cấu hình để bảo vệ ảnh đã xử lý. Việc xem thumbnail từ giao diện được thiết kế thông qua URL ký sẵn, phù hợp với mô hình lưu trữ riêng tư.




*Hình 3.9. Bucket đầu ra đã bật chặn truy cập công khai*

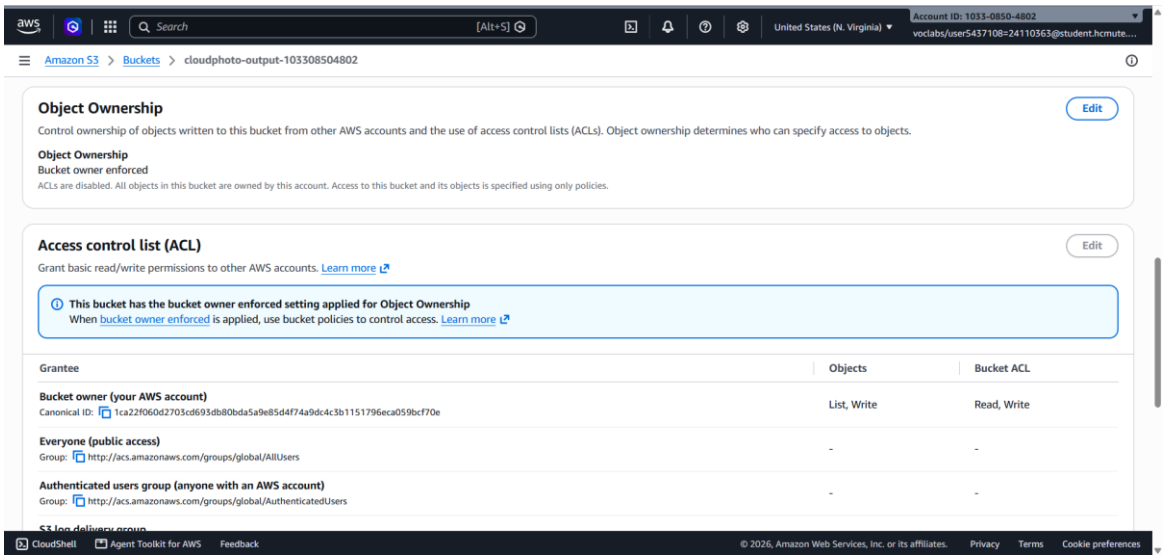
### 3.5.2. Quản lý quyền sở hữu đối tượng

Trong mục Object Ownership, nhóm sử dụng Bucket owner enforced cho bucket đầu vào. Khi đó, ACL bị vô hiệu hóa và quyền truy cập được quản lý bằng các chính sách của AWS, giúp thống nhất cách kiểm soát ảnh tải lên.




*Hình 3.10. Cấu hình quyền sở hữu đối tượng của bucket đầu vào*

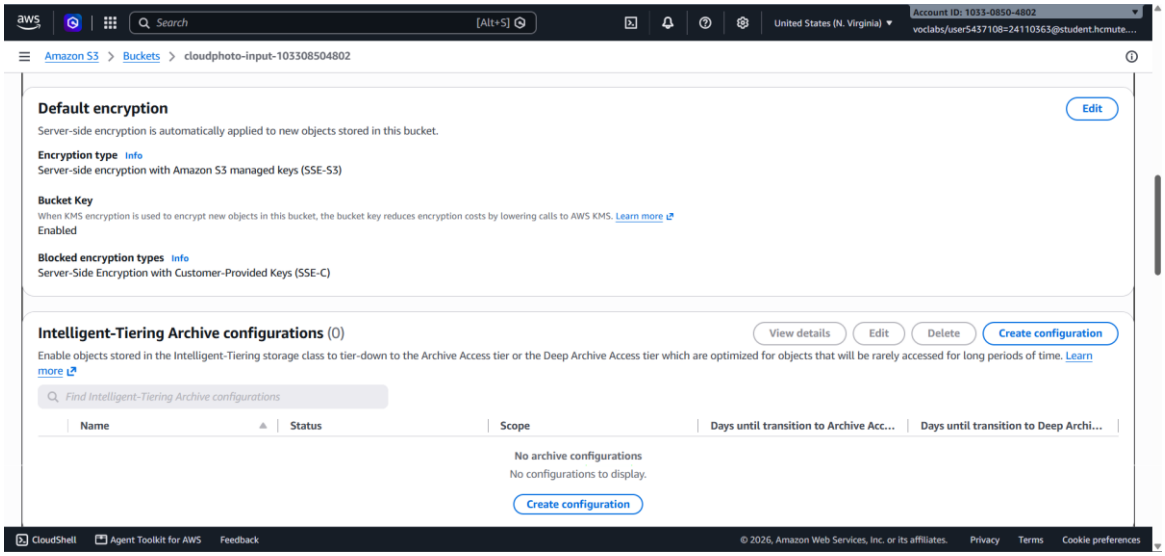
Bucket đầu ra cũng sử dụng Bucket owner enforced. Như vậy, cả ảnh gốc và thumbnail đều được quản lý theo cùng một cơ chế quyền sở hữu và không dùng ACL riêng lẻ cho từng tệp.




*Hình 3.11. Cấu hình quyền sở hữu đối tượng của bucket đầu ra*

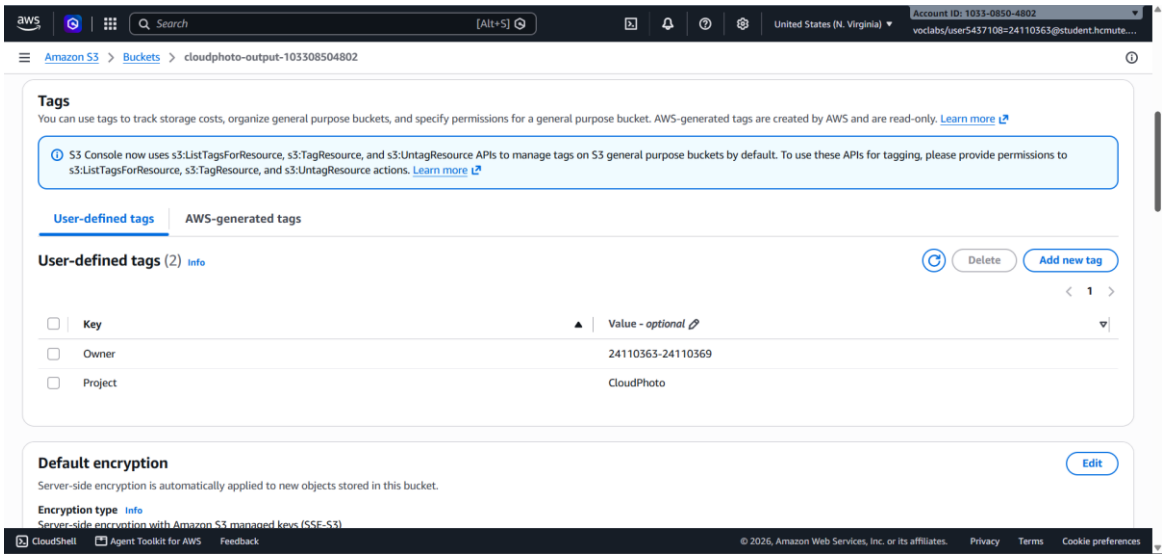
### 3.5.3. Mã hóa và gắn nhãn tài nguyên

Nhóm lựa chọn mã hóa mặc định SSE-S3 trong mục Properties. Cơ chế này sử dụng khóa do Amazon S3 quản lý để mã hóa dữ liệu khi lưu trữ. Hình dưới xác nhận bucket đầu vào đã sử dụng SSE-S3.




*Hình 3.12. Mã hóa mặc định SSE-S3 của bucket đầu vào*

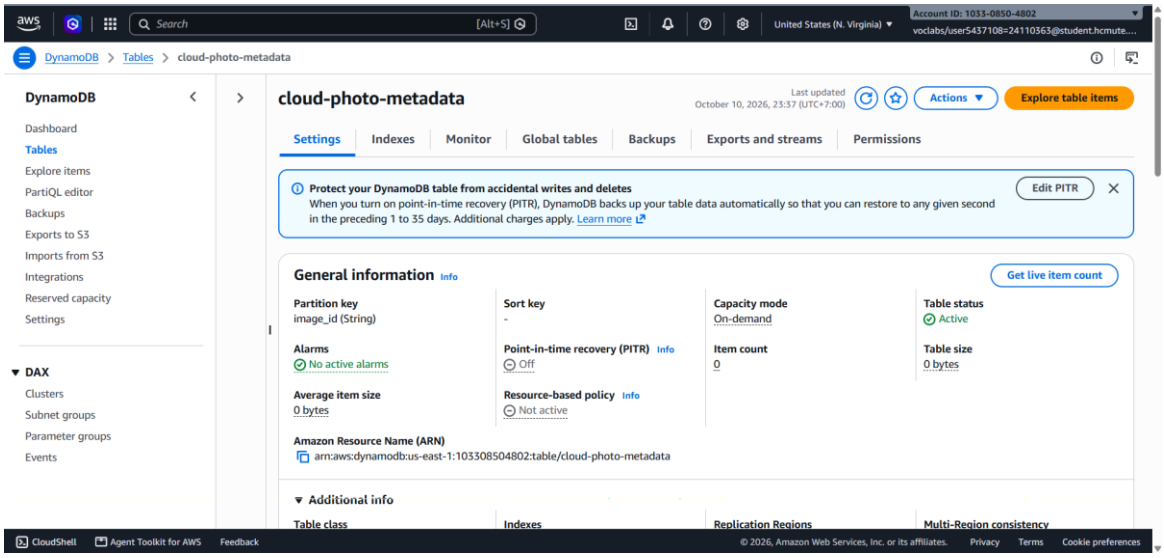
Nhóm áp dụng SSE-S3 cho bucket đầu ra và gắn nhãn Project=CloudPhoto, Owner=24110363-24110369 cho hai bucket. Các nhãn giúp nhận diện tài nguyên thuộc đề tài và thành viên phụ trách. Màn hình dưới thể hiện nhãn quản lý và loại mã hóa của bucket đầu ra.




*Hình 3.13. Nhãn quản lý và mã hóa của bucket đầu ra*

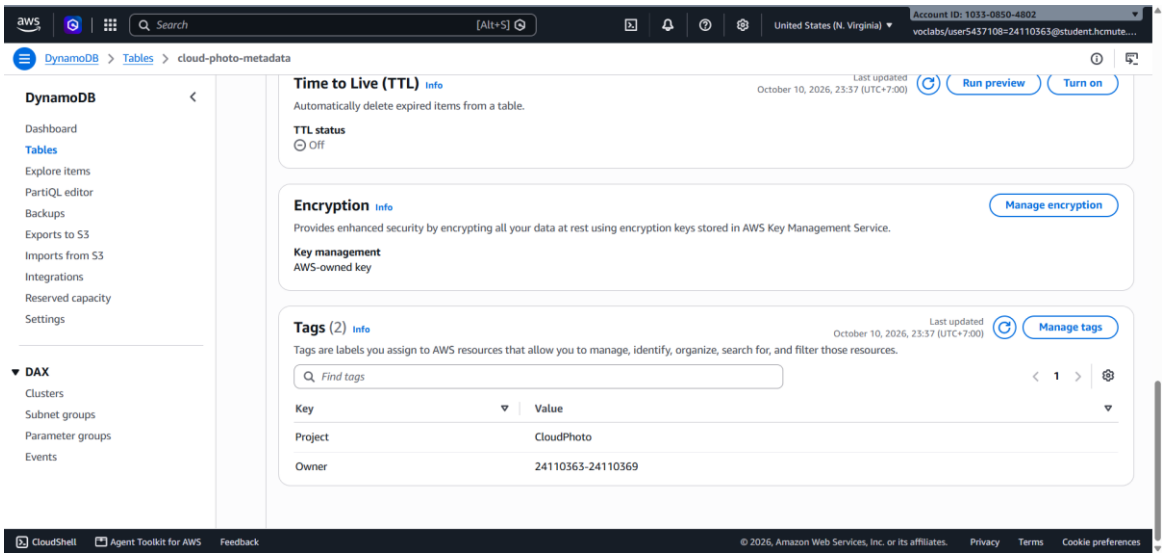
## 3.6. Tạo bảng lưu thông tin xử lý ảnh

Trong DynamoDB, nhóm tạo bảng cloud-photo-metadata, đặt khóa phân vùng image_id kiểu String và không sử dụng khóa sắp xếp. Bảng dùng chế độ On-demand để đáp ứng số lượng yêu cầu thay đổi trong quá trình thử nghiệm. Trạng thái Active xác nhận bảng đã sẵn sàng; số lượng bản ghi đang hiển thị bằng 0.




*Hình 3.14. Bảng DynamoDB đã hoạt động với khóa image_id*

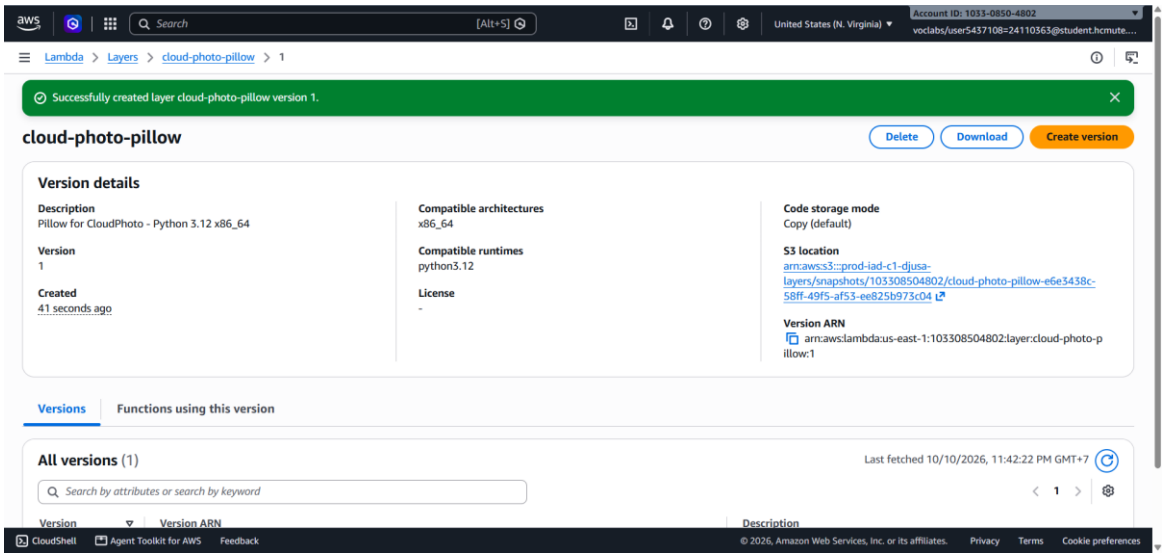
Nhóm gắn nhãn Project và Owner cho bảng để thống nhất với các bucket S3. Bảng sử dụng khóa mã hóa do AWS sở hữu; TTL đang tắt để các bản ghi không bị xóa theo thời hạn tự động. Bảng sẽ lưu thông tin ảnh gốc, các thumbnail và trạng thái khi worker được tích hợp.




*Hình 3.15. Cấu hình mã hóa và nhãn quản lý của bảng DynamoDB*

## 3.7. Chuẩn bị thư viện tạo thumbnail

Trong Lambda, nhóm chọn Layers, tạo layer cloud-photo-pillow và tải lên gói ZIP chứa thư viện Pillow dành cho Python 3.12 trên Linux x86_64. Nhóm khai báo runtime Python 3.12 và kiến trúc x86_64; kết quả đã tạo thành công phiên bản 1. Layer này cung cấp thư viện đọc ảnh, thay đổi kích thước và thêm watermark cho hàm xử lý.

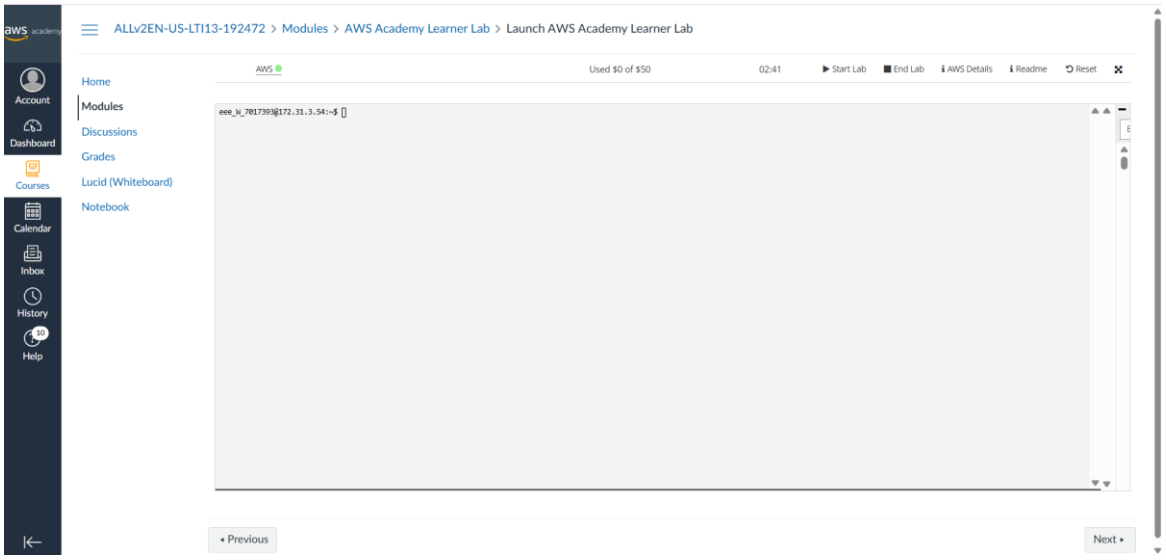



*Hình 3.16. Pillow Layer phiên bản 1 dành cho Python 3.12 và x86_64*

## 3.8. Theo dõi ngân sách thực hành

Sau khi tạo các tài nguyên, nhóm kiểm tra lại ngân sách trên Learner Lab. Màn hình hiển thị Used $0 of $50, nghĩa là mức sử dụng đang được hệ thống ghi nhận là

0 USD trên hạn mức 50 USD. Đây là số liệu tại thời điểm quan sát và có thể cập nhật trễ.




*Hình 3.17. Ngân sách Learner Lab sau khi thiết lập tài nguyên*

# CHƯƠNG 4. TỔ CHỨC THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ

## 4.1. Phân công công việc

Trong tuần 1, công việc được phân chia giữa hai thành viên theo các nhóm nhiệm vụ dưới đây. Thời lượng thể hiện kế hoạch dự kiến cho từng nhiệm vụ.

*Bảng 4.1. Phân công và thời gian dự kiến của tuần 1*

| **Công việc chi tiết** | **Thành viên phụ trách** | **Thời gian dự kiến** |
|---|---|---|
| Tìm hiểu S3, Lambda và quy trình tạo thumbnail theo sự kiện | Bùi Nguyễn Duy Trung | 2 giờ |
| Thiết kế kiến trúc, lựa chọn Region và kiểm tra LabRole | Bùi Nguyễn Duy Trung | 2 giờ |
| Tạo hai bucket S3, thiết lập bảo mật và bảng DynamoDB | Bùi Nguyễn Duy Trung | 2 giờ |
| Tạo Pillow Layer; kiểm tra runtime và kiến trúc xử lý | Bùi Nguyễn Duy Trung | 2 giờ |
| Phân tích yêu cầu, phạm vi và tiêu chí đầu ra của ảnh | Đỗ Anh Tuấn | 2 giờ |
| Chuẩn bị Python, Git, môi trường ảo và chạy kiểm thử | Đỗ Anh Tuấn | 2 giờ |
| Rà soát luồng tải ảnh, mã xử lý và bộ kiểm thử | Đỗ Anh Tuấn | 2 giờ |
| Tổng hợp kết quả thực hành, hình ảnh và báo cáo | Đỗ Anh Tuấn | 2 giờ |

Tổng thời gian dự kiến của Bùi Nguyễn Duy Trung là 8 giờ và Đỗ Anh Tuấn là 8 giờ.

## 4.2. Kết quả đạt được

*Bảng 4.2. Kết quả thực hiện trong tuần 1*

| **Hạng mục** | **Kết quả** | **Ý nghĩa đối với đề tài** |
|---|---|---|
| Thiết kế hệ thống | Xác định luồng S3, Lambda, Pillow và DynamoDB | Làm cơ sở triển khai quy trình tạo thumbnail tự động |

| | | |
|---|---|---|
| Môi trường phát triển | Python 3.12.10; Git 2.54.0; 19/19 kiểm thử đạt | Sẵn sàng phát triển và kiểm tra mã nguồn |
| Role thực thi | Đã kiểm tra LabRole và chính sách tin cậy cho Lambda | Chuẩn bị danh tính thực thi cho hàm xử lý ảnh |
| Lưu trữ S3 | Hai bucket riêng tư; có uploads/; bật SSE-S3 | Tách biệt, bảo vệ ảnh gốc và ảnh thu nhỏ |
| Bảng DynamoDB | Active; khóa image_id; chế độ On-demand | Sẵn sàng lưu metadata và trạng thái xử lý |
| Thư viện Pillow | Layer phiên bản 1; Python 3.12; x86_64 | Chuẩn bị các chức năng xử lý ảnh cho Lambda |

### 4.3. Đánh giá và kết luận

Tuần 1 đã hoàn thành nền tảng cần thiết cho đề tài xử lý ảnh tự động khi tải lên S3. Các tài nguyên được tạo theo kiến trúc đã lựa chọn, cấu hình bảo mật được kiểm tra và bộ kiểm thử cục bộ đạt 19/19 trường hợp. Kết quả này đáp ứng mục tiêu chuẩn bị môi trường và hạ tầng.

Trong phạm vi tuần 1, việc tạo thumbnail tự động trên AWS chưa được triển khai hoàn chỉnh. Các bước tích hợp tiếp theo cần kiểm tra việc Lambda nhận sự kiện S3, sử dụng Pillow Layer, ghi ảnh kết quả và cập nhật metadata. Đây là cơ sở để đánh giá toàn bộ quy trình xử lý ảnh ở giai đoạn tiếp theo.

### 4.4. Khai báo sử dụng AI và minh chứng hoàn thiện

Nhóm sử dụng ChatGPT Sol 6.1 hỗ trợ chủ yếu phần mã nguồn và hướng dẫn cấu hình; sinh viên trực tiếp thực hiện cấu hình trên máy và AWS Academy. Google Gemini hỗ trợ tóm lược/rà soát commit, hướng dẫn push GitHub và trình bày Markdown. Bản khai báo bổ sung ghi rõ hai phạm vi này cùng phần nhóm hoàn thiện và link commit đối chiếu.

Thông tin công cụ được cập nhật theo xác nhận của nhóm ngày 15/10/2026. Các prompt dưới đây là bản tóm lược yêu cầu đã trao đổi, không giả định là bản chép nguyên văn hội thoại. Model ChatGPT ghi Sol 6.1 theo xác nhận của nhóm; phiên bản Gemini không được suy đoán khi chưa ghi nhận chính xác.

#### 4.4.1. ChatGPT — Hỗ trợ mã nguồn, hướng dẫn cấu hình và rà soát báo cáo

Công cụ AI: ChatGPT. Phiên bản/model: Sol 6.1 (theo xác nhận của nhóm).

Prompt đã dùng (tóm lược theo xác nhận của nhóm): Hướng dẫn xây dựng hệ thống tạo thumbnail trên AWS bằng S3, Lambda, DynamoDB và Pillow; hỗ trợ mã nguồn xử lý ảnh, web và kiểm thử, hướng dẫn cấu hình từng bước để nhóm thực hiện trên máy và AWS Academy.

Nội dung AI tạo ra/hỗ trợ: Hướng dẫn kiến trúc và cấu hình, đề xuất/soạn mã Python xử lý ảnh, Lambda worker, giao diện Flask và các thành phần kiểm thử; hỗ trợ tổ chức tài liệu và rà soát hồ sơ nộp lại. Đây là công cụ hỗ trợ phần code và hướng dẫn cấu hình chủ yếu theo xác nhận của nhóm.

Phần sinh viên đã sửa/hoàn thiện:

Bùi Nguyễn Duy Trung: Trực tiếp thực hiện các bước cấu hình trên máy và AWS Academy: Python/Git, hai bucket S3, bảo mật bucket, LabRole, DynamoDB và Pillow Layer. Khôi phục 24 ảnh gốc cùng danh mục; 17 ảnh khớp SHA-256 với ảnh trong Word. Commit này ghi nhận việc khôi phục minh chứng, không phải ngày tạo tài nguyên.

Minh chứng: Commit 10b9d98 — evidence/tuan-1/README.md

Bùi Nguyễn Duy Trung: Đối chiếu tên tài nguyên và thông số thật: cloudphoto-input-103308504802, cloudphoto-output-103308504802, bảng cloud-photo-metadata chỉ

có khóa image_id; Python 3.12 và x86_64 của layer. Ghi lại cấu hình và chuẩn bị script đóng gói.

Minh chứng: Commit 213b093 — config/aws-resources.json

Bùi Nguyễn Duy Trung: Áp dụng và hoàn thiện mã Pillow theo yêu cầu dự án: JPEG/PNG tĩnh, giới hạn 10 MiB/24 triệu pixel, cạnh dài tối đa 150/400/1000 px, giữ tỷ lệ, ghép nền trắng cho ảnh trong suốt, watermark CLOUD PHOTO và loại EXIF. Mã có AI hỗ trợ; commit được dẫn để đối chiếu phiên bản hoàn thiện.

Minh chứng: Commit 1c41d75 — src/processor/image_tools.py

Bùi Nguyễn Duy Trung: Hoàn thiện worker với cấu hình bucket/prefix, ba ảnh đầu ra và metadata trạng thái; kiểm tra các nhánh sự kiện trùng/lỗi qua kiểm thử giả lập. Chưa khai báo đã tích hợp đầy đủ Lambda trên AWS trong tuần 1.

Minh chứng: Commit e50cb0c — src/processor/lambda_function.py

Đỗ Anh Tuấn: Theo phân công trong repo, hoàn thiện phần Flask có AI hỗ trợ: đăng nhập, CSRF, tải ảnh bằng tên khóa duy nhất, gallery và header bảo mật; đối chiếu bằng kiểm thử web.

Minh chứng: Commit 9b0aca9 — src/web/app.py

Đỗ Anh Tuấn: Hoàn thiện 10 ca ảnh: bốn hình dạng, RGBA, ba đầu vào không hợp lệ, GIF và EXIF; kiểm tra các kích thước và tỷ lệ đầu ra.

Minh chứng: Commit 30838ce — tests/test_images.py

Đỗ Anh Tuấn: Hoàn thiện năm ca worker dùng mock S3/DynamoDB và bốn ca Flask; cấu hình kiểm thử tự động. Log CI của bản f92ab84 ghi nhận 19/19 ca đạt, không phải kiểm thử AWS tích hợp.

Minh chứng: Commit f3e62ba — tests/test_worker.py

#### 4.4.2. Google Gemini — Tóm lược, rà soát commit và hướng dẫn trình bày Markdown

Công cụ AI: Google Gemini. Phiên bản/model: Không ghi nhận chính xác phiên bản/model.

Prompt đã dùng (tóm lược theo xác nhận của nhóm): Xem và tóm lược lịch sử commit của dự án, kiểm tra commit/phân công đã phù hợp chưa, hướng dẫn push lên GitHub và cách viết file Markdown trình bày nội dung nộp tuần 1.

Nội dung AI tạo ra/hỗ trợ: Gợi ý cách rà soát và tóm lược commit, hướng dẫn thao tác push GitHub và bố cục Markdown cho báo cáo. Không kê khai Gemini là công cụ chủ yếu sinh code hoặc viết các bài kiểm thử.

Phần sinh viên đã sửa/hoàn thiện:

Nhóm sinh viên: Đối chiếu từng link commit với tệp thật và trách nhiệm thành viên; giữ 13 commit mã nguồn tuần 1 làm mốc, bổ sung riêng commit khôi phục ảnh. Các commit mới ghi đúng thời điểm khôi phục/biên tập, không sửa lịch sử cũ.

Minh chứng: Commit 10b9d98 — evidence/tuan-1/DOI_CHIEU_ANH.json

Nhóm sinh viên: Thống nhất lại phạm vi sử dụng AI theo xác nhận thực tế; trình bày phần hỗ trợ, phần hoàn thiện và link minh chứng trong Markdown. Bản bổ sung có ChatGPT hỗ trợ rà soát và biên tập; nhóm kiểm tra lại thông tin trước khi nộp.

Minh chứng: Commit 27b1e83 — AI_DISCLOSURE.md

#### 4.4.3. Danh mục ảnh cấu hình và phạm vi minh chứng

Nhóm khôi phục 24 ảnh gốc và danh mục từ bản lưu cục bộ. Đối chiếu SHA-256 xác nhận 17 ảnh trùng với ảnh minh chứng trong báo cáo; các ảnh bổ trợ được giữ và ghi rõ trong danh mục. Không tạo ảnh AWS bằng AI, không tự bổ sung ngày chụp hoặc người chụp chưa được xác nhận.

Danh mục ảnh: 24 ảnh gốc và vị trí báo cáo — Danh mục CSV

Commit khôi phục minh chứng: 10b9d98

Commit xác nhận phạm vi AI và trình bày Markdown: 27b1e83

Ngày commit khôi phục chỉ ghi nhận việc đưa các ảnh có sẵn trở lại repo, không phải ngày cấu hình AWS hoặc ngày chụp. Những ảnh bucket output và bảng DynamoDB chưa có dữ liệu phản ánh trạng thái chuẩn bị tài nguyên tuần 1.

Các commit được dẫn ghi nhận phiên bản mã nguồn nhóm đã hoàn thiện và đưa vào repo. Lịch sử hiện tại chủ yếu là các commit thêm file; repo không lưu riêng bản nháp

AI để so sánh trước–sau. Vì vậy, link commit dùng để đối chiếu nội dung, người đứng tên commit và kết quả kiểm thử; không dùng để khẳng định Git tự chứng minh từng dòng do AI hay sinh viên viết.

Lần chạy GitHub Actions tại commit f92ab842641c61cba4c7c790ba85613f33ca26ae ghi nhận 19 passed trên Python 3.12, gồm 10 ca ảnh, 5 ca worker và 4 ca Flask. S3/DynamoDB trong các bài thử được giả lập; kết quả này chưa chứng minh luồng S3 → Lambda → S3/DynamoDB đã chạy hoàn chỉnh trên AWS. Ảnh cấu hình AWS trong báo cáo được giữ để minh chứng phần tài nguyên đã chuẩn bị.

Minh chứng kiểm thử: GitHub Actions — 19/19 kiểm thử đạt

Kho mã nguồn công khai: bndt0103/aws-thumbnail — Lịch sử commit

Nhóm chịu trách nhiệm đối chiếu khai báo công cụ và prompt với quá trình làm việc thực tế; kiểm tra mã nguồn, kết quả kiểm thử và thông tin cấu hình trước khi nộp. AI hỗ trợ đề xuất và rà soát; kết quả tài nguyên AWS và phạm vi hoàn thành được trình bày theo minh chứng của tuần 1.

## TÀI LIỆU THAM KHẢO


1. Amazon Web Services. Creating a general purpose bucket.
2. Amazon Web Services. Controlling ownership of objects and disabling ACLs.
3. Amazon Web Services. Create a table in DynamoDB.
4. Amazon Web Services. Working with layers for Python Lambda functions.
5. Nhóm Serverless. Kho mã nguồn aws-thumbnail và các commit minh chứng. https://github.com/bndt0103/aws-thumbnail. Đối chiếu ngày 15/10/2026.
6. GitHub Actions. Kết quả chạy 19 ca kiểm thử ở phiên bản f92ab84. https://github.com/bndt0103/aws-thumbnail/actions/runs/38563088053. Đối chiếu ngày 15/10/2026.